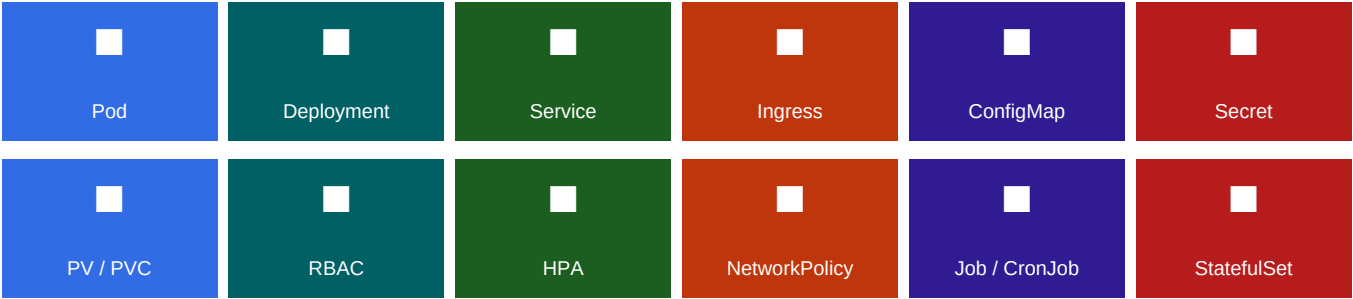


KUBERNETES

Guide Complet des Manifests YAML

Structure, champs, exemples annotés et bonnes pratiques

Pod · Deployment · Service · Ingress · ConfigMap · Secret · PersistentVolume · RBAC · HPA · NetworkPolicy · Job · CronJob · StatefulSet



Ce guide documente la structure complète de chaque manifest Kubernetes : tous les champs YAML avec leur type, leur caractère obligatoire ou optionnel, et leur rôle. Chaque objet est illustré par un exemple annoté prêt à l'emploi et accompagné de bonnes pratiques de production.

PARTIE 1 — Structure Commune à Tous les Manifests

Chaque manifest Kubernetes partage 4 champs racines obligatoires. Cette section explique leur rôle et les valeurs possibles.

1.1 — Les 4 champs racines

```
# Tout manifest Kubernetes commence par ces 4 champs
apiVersion: apps/v1 # Version de l'API K8s (voir 1.2)
kind: Deployment # Type d'objet (Pod, Service, Deployment...)
metadata: # Identité de l'objet
  name: mon-app # Nom unique dans le namespace
  namespace: production # Namespace (default si omis)
  labels: # Étiquettes clé-valeur pour sélecteurs
app: mon-app
version: "1.0"
environment: production
annotations: # Métadonnées libres (non utilisées comme sélecteurs)
description: "Service principal de l'application"
contact: "team@example.com"
spec: # Spécification de l'état désiré (varie par objet)
# ... contenu spécifique à chaque kind
```

1.2 — API Versions les plus utilisées

apiVersion	Objets couverts	Stabilité
v1	Pod, Service, ConfigMap, Secret, Namespace, PersistentVolume, PersistentVolumeClaim, ServiceAccount	GA (stable)
apps/v1	Deployment, ReplicaSet, StatefulSet, DaemonSet	GA (stable)
networking.k8s.io/v1	Ingress, NetworkPolicy	GA (stable)
batch/v1	Job, CronJob	GA (stable)
autoscaling/v2	HorizontalPodAutoscaler (HPA)	GA (stable)
rbac.authorization.k8s.io/v1	Role, ClusterRole, RoleBinding, ClusterRoleBinding	GA (stable)
storage.k8s.io/v1	StorageClass, VolumeAttachment	GA (stable)
policy/v1	PodDisruptionBudget (PDB)	GA (stable)
cert-manager.io/v1	Certificate, ClusterIssuer, Issuer (CRD cert-manager)	CRD (cert-manager)
argoproj.io/v1alpha1	Application, AppProject (CRD ArgoCD)	CRD (ArgoCD)

■ Vérifier les apiVersions disponibles sur votre cluster : `kubectl api-resources --verbs=list | grep -i kubectl api-versions`

POD — Unité de base d'exécution

Le Pod est l'objet le plus fondamental : 1 à N conteneurs partageant le même réseau et espace de stockage. En pratique, on ne crée jamais de Pod directement — on utilise un Deployment. Mais comprendre le spec.template.spec d'un Pod est indispensable.

Pod	Pod
-----	-----

apiVer

[illegible]

Champ YAML	Type	Req.	Description
<code>spec.serviceAccountName</code>	string	—	ServiceAccount K8s lié au pod. Définit les permissions RBAC. Défaut : 'default'
<code>spec.securityContext</code>	object	—	Sécurité au niveau pod : runAsUser, runAsGroup, fsGroup, seccompProfile
<code>spec.containers[].image</code>	string	✓	Image Docker. Toujours préciser un tag fixe (jamais latest en prod)
<code>spec.containers[].resources</code>	object	✓	requests = garanti, limits = maximum. OBLIGATOIRE pour HPA et scheduler
<code>spec.containers[].livenessProbe</code>	object	—	K8s redémarre le conteneur si cette probe échoue
<code>spec.containers[].readinessProbe</code>	object	—	K8s retire le pod du LoadBalancer si cette probe échoue
<code>spec.containers[].startupProbe</code>	object	—	Désactive liveness/readiness pendant le démarrage lent d'une app
<code>spec.containers[].securityContext</code>	object	—	allowPrivilegeEscalation:false, readOnlyRootFilesystem:true recommandés
<code>spec.volumes</code>	[]object	—	Liste de volumes (configMap, secret, pvc, emptyDir, hostPath...)

Champ YAML	Type	Req.	Description
<code>spec.initContainers</code>	<code>[]object</code>	—	Conteneurs exécutés séquentiellement AVANT les containers principaux
<code>spec.imagePullSecrets</code>	<code>[]object</code>	—	Secrets pour l'authentification au registry Docker privé
<code>spec.tolerations</code>	<code>[]object</code>	—	Permet au pod de se scheduler sur des nœuds avec taints
<code>spec.affinity</code>	<code>object</code>	—	Règles d'affinité/anti-affinité pour la répartition des pods

DEPLOYMENT — Gestion du cycle de vie des pods

Le Deployment est l'objet de référence pour déployer des applications stateless. Il gère les ReplicaSets, orchestre les rolling updates et permet les rollbacks.

Deployment

Deployment

apiVer

```
apiVersion: apps/v1 kind: Deployment metadata: name: mon-app namespace: production labels: app: mon-app spec: replicas: 3 # Nombre de pods désirés # Sélecteur (DOIT correspondre aux labels du template) selector: matchLabels: app: mon-app # Stratégie de mise à jour strategy: type: RollingUpdate # RollingUpdate | Recreate rollingUpdate: maxUnavailable: 1 # Max pods indisponibles pendant update maxSurge: 1 # Max pods en excès pendant update # Conserver l'historique pour les rollbacks revisionHistoryLimit: 5 # Timeout d'un déploiement (en secondes) progressDeadlineSeconds: 600 # Template de pod (identique à spec d'un Pod) template: metadata: labels: app: mon-app # DOIT correspondre à selector.matchLabels version: "1.2.0" spec: # → Voir section Pod pour tous les champs disponibles ici containers: - name: mon-app image: registry.example.com/mon-app:1.2.0 ports: - containerPort: 8080 resources: requests: cpu: 250m memory: 256Mi limits: cpu: 500m memory: 512Mi livenessProbe: httpGet: path: /health port: 8080 initialDelaySeconds: 30 periodSeconds: 15 readinessProbe: httpGet: path: /ready port: 8080 initialDelaySeconds: 10 periodSeconds: 5 securityContext: allowPrivilegeEscalation: false readOnlyRootFilesystem: true runAsNonRoot: true runAsUser: 1000
```

Commande kubectl	Description
kubectl apply -f deployment.yaml	Créer ou mettre à jour
kubectl rollout status deploy/mon-app -n prod	Suivre le déploiement en cours
kubectl rollout history deploy/mon-app	Voir l'historique des révisions
kubectl rollout undo deploy/mon-app	Rollback à la révision précédente
kubectl rollout undo deploy/mon-app --to-revision=2	Rollback à une révision spécifique
kubectl scale deploy/mon-app --replicas=5	Scaler manuellement
kubectl set image deploy/mon-app app=image:v2	Mettre à jour l'image sans modifier le YAML

SERVICE — Exposition et découverte réseau

Le Service crée un point d'accès stable (IP virtuelle + DNS) vers un groupe de pods sélectionnés par labels. Il persiste même si les pods sont recréés.

```
# ■■■ ClusterIP (accès interne uniquement) ■■■■ apiVersion: v1 kind: Service metadata: name: mon-app-svc namespace: production annotations: service.beta.kubernetes.io/azure-load-balancer-internal: "true" # LB interne Azure spec: type: ClusterIP # ClusterIP | NodePort | LoadBalancer | ExternalName selector: app: mon-app # Sélectionne les pods avec ce label ports: - name: http protocol: TCP port: 80 # Port exposé par le Service targetPort: 8080 # Port du conteneur (ou nom de port) - name: https protocol: TCP port: 443 targetPort: 8443 sessionAffinity: None # None | ClientIP (sticky sessions) --- # ■■■ LoadBalancer (accès externe via Azure LB) ■■■■ apiVersion: v1 kind: Service metadata: name: mon-app-lb namespace: production annotations: service.beta.kubernetes.io/azure-load-balancer-health-probe-request-path: /health spec: type: LoadBalancer selector: app: mon-app ports: - port: 80 targetPort: 8080 loadBalancerIP: "" # IP statique Azure (optionnel) --- # ■■■ ExternalName (alias DNS vers service externe) ■■■■ apiVersion: v1 kind: Service metadata: name: postgres-svc namespace: production spec: type: ExternalName externalName: mydb.postgres.database.azure.com # DNS cible ports: - port: 5432
```

Type	Accessible depuis	Cas d'usage typique
ClusterIP	À l'intérieur du cluster uniquement	Communication inter-services (microservices)
NodePort	Depuis l'extérieur via IP nœud + port	Dev/test, pas recommandé en prod
LoadBalancer	Depuis l'extérieur via IP publique LB	Exposition directe (alternative à Ingress)
ExternalName	DNS alias vers une ressource externe	Accès à une BDD Azure, API externe
Headless (clusterIP: None)	DNS par pod individuel	StatefulSets, découverte de pods individuels

■ **DNS interne K8s** : `..svc.cluster.local` → résout vers la ClusterIP du service. Exemple : `postgres-svc.production.svc.cluster.local:5432`

INGRESS — Routage HTTP/HTTPS entrant

L'Ingress expose des services HTTP/HTTPS via des règles de routage (host + path). Nécessite un Ingress Controller déployé dans le cluster (NGINX, AGIC, Traefik...).

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: mon-app-ingress namespace: production annotations: #
■■ cert-manager (TLS automatique Let's Encrypt) ■■■■■■■■ cert-manager.io/cluster-issuer: "letsencrypt-prod" # ■■
Annotations NGINX ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ nginx.ingress.kubernetes.io/ssl-redirect: "true"
nginx.ingress.kubernetes.io/force-ssl-redirect: "true" nginx.ingress.kubernetes.io/proxy-body-size: "50m"
nginx.ingress.kubernetes.io/proxy-read-timeout: "120" nginx.ingress.kubernetes.io/rate-limit: "100"
nginx.ingress.kubernetes.io/enable-cors: "true" nginx.ingress.kubernetes.io/cors-allow-origin:
"https://app.example.com" # ■■ Annotations AGIC (Azure Application Gateway) ■■■■■■■■ #
kubernetes.io/ingress.class: azure/application-gateway # appgw.ingress.kubernetes.io/ssl-redirect: "true" spec:
ingressClassName: nginx # Choisir le controller Ingress # TLS : certificat stocké dans un Secret K8s tls: - hosts: -
api.example.com - app.example.com secretName: example-tls # Secret créé par cert-manager rules: # ■■ Routage par
host ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ - host: api.example.com http: paths: - path: /v1/vessels # Path
exact pathType: Prefix # Prefix | Exact | ImplementationSpecific backend: service: name: vessel-svc port: number: 80
- path: /v1/classifications pathType: Prefix backend: service: name: classification-svc port: number: 80 - host:
app.example.com http: paths: - path: / pathType: Prefix backend: service: name: frontend-svc port: number: 80
```

Champ YAML	Type	Req.	Description
spec.ingressClassName	string	—	Sélectionne le controller Ingress (nginx, azure/application-gateway...). Remplace l'annotation kubernetes.io/ingress.class
spec.tls	[]object	—	Liste des hostnames TLS et du Secret contenant le certificat (créé par cert-manager)
spec.rules[].host	string	—	Hostname de la règle. Si omis : toutes les requêtes matchent
spec.rules[].http.paths[].path	string	✓	Chemin URL. Avec type Prefix : /api match /api/v1, /api/v2...
spec.rules[].http.paths[].pathType	string	✓	Prefix (recommandé), Exact (correspondance stricte), ImplementationSpecific
spec.rules[].http.paths[].backend.service	object	✓	Service K8s cible + port

CONFIGMAP — Configuration non-sensible

Le ConfigMap stocke des données de configuration non-sensibles (URLs, flags, fichiers de config) injectables dans les pods comme variables d'environnement ou volumes montés.

```
apiVersion: v1 kind: ConfigMap metadata: name: app-config namespace: production data: # ■■ Paires clé-valeur simples (variables d'env) ■■■■■■■■■■ APP_ENV: "production" LOG_LEVEL: "info" DB_HOST: "postgres-svc.production.svc.cluster.local" DB_PORT: "5432" DB_NAME: "appdb" REDIS_HOST: "redis-svc.production.svc.cluster.local" MAX_CONNECTIONS: "100" # ■■ Fichiers de configuration (multiline) ■■■■■■■■■■ app.properties: | log.level=INFO cache.enabled=true cache.ttl=3600 feature.newui=false nginx.conf: | server { listen 80; location /health { return 200; } location / { proxy_pass http://localhost:8080; } } # JSON / YAML inline config.json: | { "environment": "production", "features": { "analytics": true, "debug": false } } binaryData: # Données binaires encodées en base64 logo.png: <base64-encoded-data>
```

■ Ne jamais stocker de mots de passe, tokens ou certificats dans un ConfigMap. Utiliser Secret ou Azure Key Vault + External Secrets Operator pour les données sensibles.

```
# ■■ Injection dans un Deployment ■■■■■■■■■■ spec: template: spec: containers: - name: app # Méthode 1 : Injecter TOUT le ConfigMap comme variables d'env envFrom: - configMapRef: name: app-config optional: false # true = ne pas échouer si le CM n'existe pas # Méthode 2 : Sélectionner une clé spécifique env: - name: LOG_LEVEL valueFrom: configMapKeyRef: name: app-config key: LOG_LEVEL # Méthode 3 : Monter comme fichiers dans un volume volumeMounts: - name: config-files mountPath: /etc/config readOnly: true volumes: - name: config-files configMap: name: app-config items: # Sélectionner des clés spécifiques - key: app.properties path: application.properties # Renommer le fichier - key: nginx.conf path: nginx.conf defaultMode: 0644 # Permissions des fichiers
```


SECRET — Données sensibles

Le Secret stocke des données sensibles encodées en base64 (mots de passe, tokens, certificats). Attention : base64 n'est PAS un chiffrement. Utiliser Azure Key Vault + External Secrets Operator en production.

[illegible]

Type	Usage	Champs data requis
Opaque	Usage général (mots de passe, tokens...)	Libre
kubernetes.io/tls	Certificats TLS	tls.crt, tls.key
kubernetes.io/dockerconfigjson	Authentification registry Docker	.dockerconfigjson
kubernetes.io/service-account-token	Token ServiceAccount	token (auto-généré)
kubernetes.io/basic-auth	Authentification HTTP basique	username, password
kubernetes.io/ssh-auth	Authentification SSH	ssh-privatekey

```
# ■■ External Secret (recommandé en PROD – depuis Azure Key Vault) ■■ apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: app-secrets namespace: production spec: refreshInterval: 1h secretStoreRef: name: azure-keyvault # ClusterSecretStore configuré pour AKV kind: ClusterSecretStore target: name: app-secrets # Secret K8s créé/synchronisé creationPolicy: Owner data: - secretKey: db-password # Clé dans le Secret K8s créé remoteRef: key: app-db-password # Nom du secret dans Azure Key Vault - secretKey: jwt-secret remoteRef: key: app-jwt-secret
```

PERSISTENTVOLUME & CLAIM — Stockage persistant

PersistentVolume (PV) représente une unité de stockage dans le cluster. PersistentVolumeClaim (PVC) est la demande de stockage faite par un pod. StorageClass permet le provisionnement dynamique.

```
# █ StorageClass (provisionnement dynamique Azure) ████████ apiVersion: storage.k8s.io/v1 kind: StorageClass  
metadata: name: managed-premium provisioner: disk.csi.azure.com parameters: skuName: Premium_LRS # Standard_LRS |  
Premium_LRS | UltraSSD_LRS cachingmode: ReadOnly reclaimPolicy: Delete # Delete | Retain volumeBindingMode:  
WaitForFirstConsumer # Provisionner seulement quand un pod demande allowVolumeExpansion: true # Autoriser  
l'agrandissement du volume --- # █ PersistentVolumeClaim ████████████████████████████████████████████████████████████ apiVersion: v1  
kind: PersistentVolumeClaim metadata: name: data-pvc namespace: production spec: accessModes: - ReadWriteOnce #  
ReadWriteOnce | ReadOnlyMany | ReadWriteMany storageClassName: managed-premium resources: requests: storage: 50Gi  
volumeMode: Filesystem # Filesystem | Block --- # █ PersistentVolume manuel (statique) ████████████████████████████████████████████████████████████  
apiVersion: v1 kind: PersistentVolume metadata: name: data-pv spec: capacity: storage: 50Gi accessModes: -  
ReadWriteOnce reclaimPolicy: Retain storageClassName: managed-premium csi: driver: disk.csi.azure.com volumeHandle:  
/subscriptions/.../resourceGroups/.../providers/Microsoft.Compute/disks/mon-disk mountOptions: - barrier=0 -  
cache=strictautoflush
```

Mode	Abrév.	Description	Cas d'usage
ReadWriteOnce	RWO	Lu/écrit par 1 seul nœud simultanément	BDD, données d'état (StatefulSet)
ReadOnlyMany	ROX	Lu par plusieurs nœuds simultanément	Données partagées en lecture seule
ReadWriteMany	RWX	Lu/écrit par plusieurs nœuds simultanément	Stockage partagé (NFS, Azure Files)
ReadWriteOncePod	RWOP	Lu/écrit par 1 seul pod (K8s 1.22+)	Isolation stricte par pod

RBAC — Contrôle d'accès basé sur les rôles

Le RBAC (Role-Based Access Control) définit QUI peut faire QUOI sur QUELLES ressources dans le cluster. 4 objets : ServiceAccount, Role, ClusterRole, RoleBinding, ClusterRoleBinding.

```
# ■ ServiceAccount (identité d'un pod) ■■■■■■■■■■■■■■■■■■■■■■ apiVersion: v1 kind: ServiceAccount metadata: name: mon-app-sa namespace: production annotations: # Workload Identity Azure (lie le SA à une Managed Identity) azure.workload.identity/client-id: "<client-id-managed-identity>" automountServiceAccountToken: false # Sécurité : désactiver si non nécessaire --- # ■ Role (permissions dans UN namespace) ■■■■■■■■■■■■■■■■■■■■■■ apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: name: pod-reader namespace: production rules: - apiGroups: [""] # "" = core API group (pods, services...) resources: ["pods", "pods/log"] verbs: ["get", "list", "watch"] # get|list|watch|create|update|patch|delete - apiGroups: ["apps"] resources: ["deployments"] verbs: ["get", "list", "watch"] resourceNames: ["mon-app"] # Restreindre à un nom de ressource spécifique --- # ■ RoleBinding (associe Role + ServiceAccount) ■■■■■■■■■■■■■■■■■■■■■■ apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: read-pods-binding namespace: production subjects: - kind: ServiceAccount name: mon-app-sa namespace: production - kind: User # Utilisateur (AAD, certificat...) name: jane@example.com apiGroup: rbac.authorization.k8s.io - kind: Group # Groupe AAD name: devops-team apiGroup: rbac.authorization.k8s.io roleRef: kind: Role name: pod-reader apiGroup: rbac.authorization.k8s.io --- # ■ ClusterRole (permissions cluster-wide) ■■■■■■■■■■■■■■■■■■■■■■ apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRole metadata: name: node-reader rules: - apiGroups: ["" ] resources: ["nodes", "namespaces"] verbs: ["get", "list", "watch"] - nonResourceURLs: ["/healthz", "/metrics"] verbs: ["get"]
```

Verbe	Équivalent HTTP	Description
get	GET /resource/name	Lire un objet spécifique
list	GET /resource	Lister tous les objets (+ watch pour streaming)
watch	GET /resource?watch	Surveiller les changements en temps réel
create	POST /resource	Créer un nouvel objet
update	PUT /resource/name	Remplacer complètement un objet
patch	PATCH /resource/name	Modifier partiellement un objet
delete	DELETE /resource/name	Supprimer un objet
deletecollection	DELETE /resource	Supprimer une collection d'objets

HPA — Autoscaling Horizontal des Pods

Le HorizontalPodAutoscaler ajuste automatiquement le nombre de replicas d'un Deployment en fonction de métriques (CPU, mémoire, métriques custom Prometheus, KEDA...).

■ Le HPA nécessite que les pods aient des `requests.cpu/memory` définis. Sans `requests`, K8s ne peut pas calculer l'utilisation relative. Vérifier avec : `kubectl top pods -n production`

```
# ■ PodDisruptionBudget (garantir la dispo pendant les opérations) ■ apiVersion: policy/v1 kind:
PodDisruptionBudget metadata: name: mon-app-pdb namespace: production spec: minAvailable: 2 # ou maxUnavailable: 1
selector: matchLabels: app: mon-app
```

NETWORKPOLICY — Isolation réseau des pods

Le *NetworkPolicy* contrôle le trafic réseau entrant (*ingress*) et sortant (*egress*) des pods. Sans *NetworkPolicy*, tous les pods peuvent communiquer entre eux librement.

```
# ■■ Bloquer tout le trafic entrant par défaut ■■■■■■■■■■ apiVersion: networking.k8s.io/v1 kind: NetworkPolicy
metadata: name: default-deny-ingress namespace: production spec: podSelector: {} # Sélectionne TOUS les pods du
namespace policyTypes: - Ingress # Ingress | Egress | [Ingress, Egress] --- # ■■ Autoriser uniquement depuis
l'Ingress Controller ■■■■■■ apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: allow-from-ingress
namespace: production spec: podSelector: matchLabels: app: mon-app # S'applique aux pods mon-app policyTypes: -
Ingress ingress: - from: - namespaceSelector: # Depuis le namespace ingress-nginx matchLabels:
kubernetes.io/metadata.name: ingress-nginx - podSelector: # ET uniquement les pods du controller matchLabels:
app.kubernetes.io/component: controller ports: - protocol: TCP port: 8080 --- # ■■ Autoriser uniquement la BDD vers
PostgreSQL ■■■■■■■■■■ apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name: allow-db-egress
namespace: production spec: podSelector: matchLabels: app: mon-app policyTypes: - Egress egress: - to: - ipBlock: #
Autoriser vers une IP externe (Azure DB) cidr: 10.2.0.0/24 # Plage IP du subnet BDD ports: - protocol: TCP port: 5432
- to: # Toujours autoriser DNS - namespaceSelector: {} ports: - protocol: UDP port: 53 - protocol: TCP port: 53
```

Sélecteur	Cible	Exemple
podSelector	Pods dans le MÊME namespace	matchLabels: {app: mon-app}
namespaceSelector	Tous les pods d'un namespace	matchLabels: {kubernetes.io/metadata.name: prod}
ipBlock	Plage d'adresses IP	cidr: 10.0.0.0/8, except: [10.1.0.0/16]
podSelector + namespaceSelector (combinés)	Pods spécifiques dans un namespace spécifique	Les deux dans le même élément 'from'

JOB & CRONJOB — Tâches ponctuelles et planifiées

Job : exécute un pod jusqu'à complétion (migration BDD, script one-shot). CronJob : exécute un Job selon une planification cron.

```
# ■ Job : tâche à exécution unique ■■■■■■■■■■■■■■■■■■■■■■■■■■ apiVersion: batch/v1 kind: Job metadata: name: db-migration namespace: production annotations: argocd.argoproj.io/hook: PreSync # Hook ArgoCD PreSync argocd.argoproj.io/hook-delete-policy: HookSucceeded spec: completions: 1 # Nombre de pods à compléter avec succès parallelism: 1 # Pods lancés en parallèle backoffLimit: 3 # Réessaie en cas d'échec activeDeadlineSeconds: 600 # Timeout global du job (10 min) ttlSecondsAfterFinished: 3600 # Supprimer le job 1h après complétion template: metadata: labels: job: db-migration spec: restartPolicy: OnFailure # OnFailure | Never (JAMAIS Always pour un Job) containers: - name: migrate image: registry.example.com/app:1.2.0 command: ["python", "manage.py", "migrate", "--noinput"] envFrom: - secretRef: name: app-secrets resources: requests: cpu: 100m memory: 128Mi --- # ■ CronJob : tâche planifiée ■■■■■■■■■■■■■■■■■■■■■■■■■■ apiVersion: batch/v1 kind: CronJob metadata: name: cleanup-old-records namespace: production spec: schedule: "0 2 * * *" # Tous les jours à 2h du matin (UTC) # Format : minute heure jour-du-mois mois jour-de-semaine # Exemples : "*" / 5 * * * = toutes les 5 min # "0 0 1 * *" = 1er du mois à minuit concurrencyPolicy: Forbid # Allow | Forbid | Replace successfulJobsHistoryLimit: 3 # Conserver 3 jobs réussis failedJobsHistoryLimit: 1 # Conserver 1 job échoué startingDeadlineSeconds: 120 # Timeout de démarrage si retard jobTemplate: spec: backoffLimit: 2 template: spec: restartPolicy: OnFailure containers: - name: cleanup image: registry.example.com/app:1.2.0 command: ["python", "scripts/cleanup.py"] envFrom: - configMapRef: name: app-config
```

STATEFULSET — Applications avec état

StatefulSet est utilisé pour les applications avec état (BDD, Kafka, Redis...). Il garantit un identifiant réseau stable, un stockage persistant par pod et un déploiement/rollout ordonné.

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: postgresql namespace: production spec: serviceName:
"postgresql-headless" # Service Headless requis replicas: 3 selector: matchLabels: app: postgresql # Politique de
mise à jour updateStrategy: type: RollingUpdate rollingUpdate: partition: 0 # Mettre à jour seulement les pods >=
partition # Politique de gestion des pods podManagementPolicy: OrderedReady # OrderedReady | Parallel template:
metadata: labels: app: postgresql spec: terminationGracePeriodSeconds: 30 containers: - name: postgresql image:
postgres:15-alpine ports: - containerPort: 5432 name: postgresql env: - name: POSTGRES_PASSWORD valueFrom:
secretKeyRef: name: pg-secret key: password - name: PGDATA value: /var/lib/postgresql/data/pgdata volumeMounts: -
name: data mountPath: /var/lib/postgresql/data resources: requests: cpu: 250m memory: 512Mi limits: cpu: 1000m
memory: 1Gi # VolumeClaimTemplate (1 PVC créé par pod automatiquement) volumeClaimTemplates: - metadata: name: data
spec: accessModes: ["ReadWriteOnce"] storageClassName: managed-premium resources: requests: storage: 20Gi --- # ■■
Service Headless (requis pour StatefulSet) ■■■■■■■■■■■■ apiVersion: v1 kind: Service metadata: name:
postgresql-headless namespace: production spec: clusterIP: None # Headless = pas de ClusterIP selector: app:
postgresql ports: - port: 5432 name: postgresql
```

■ Les pods d'un StatefulSet ont des noms stables : *postgresql-0*, *postgresql-1*, *postgresql-2*. Le DNS de chaque pod est : *postgresql-0.postgresql-headless.production.svc.cluster.local*

SYNTHÈSE — Récapitulatif et Bonnes Pratiques

Vue d'ensemble de tous les objets Kubernetes et checklist des bonnes pratiques de production.

Tous les objets Kubernetes — récapitulatif

Objet	API	Utilisation	Caractéristique clé
Pod	v1	Unité d'exécution de base	Ne jamais créer directement en prod
Deployment	apps/v1	Applications stateless scalables	Rolling update, rollback, replicas
StatefulSet	apps/v1	Applications avec état (BDD, Kafka...)	Pods ordonnés, PVC par pod, DNS stable
DaemonSet	apps/v1	1 pod par nœud (monitoring, log...)	Suit l'ajout/suppression de nœuds
Job	batch/v1	Tâche à exécution unique (migration...)	restartPolicy: OnFailure Never
CronJob	batch/v1	Tâche planifiée (format cron)	Crée un Job à chaque exécution
Service	v1	Exposition et découverte réseau	Stable même si les pods changent
Ingress	networking.k8s.io/v1	Routage HTTP/HTTPS entrant	Nécessite un Ingress Controller
ConfigMap	v1	Configuration non-sensible	Env vars ou fichiers montés
Secret	v1	Données sensibles (base64)	Utiliser AKV + ESO en production
PV / PVC	v1	Stockage persistant	PVC = demande, PV = resource
StorageClass	storage.k8s.io/v1	Provisionnement dynamique de stockage	managed-premium pour Azure Disk
ServiceAccount	v1	Identité d'un pod (RBAC)	1 SA par application recommandé
Role	rbac.../v1	Permissions dans 1 namespace	Toujours privileged minimum
ClusterRole	rbac.../v1	Permissions cluster-wide	Réserver aux composants système
RoleBinding	rbac.../v1	Lie un Role à un SA/User/Group	Scope: 1 namespace
NetworkPolicy	networking.k8s.io/v1	Isolation réseau entre pods	Deny all par défaut recommandé
HPA	autoscaling/v2	Autoscaling horizontal	Requiert metrics-server
PDB	policy/v1	Garantit la dispo lors maintenance	minAvailable ou maxUnavailable
LimitRange	v1	Limites par défaut par namespace	Protège contre pods sans limits
ResourceQuota	v1	Quotas de ressources par namespace	CPU/mem/pods/services max

Checklist Production — 15 points essentiels

■	resources.requests ET limits définis sur TOUS les conteneurs
■	livenessProbe ET readinessProbe configurées sur chaque conteneur
■	startupProbe pour les applications à démarrage lent (> 30s)
■	securityContext: runAsNonRoot:true, allowPrivilegeEscalation:false, readOnlyRootFilesystem:true
■	capabilities: drop: [ALL] dans le securityContext du conteneur
■	ServiceAccount dédié par application (pas 'default')

■	automountServiceAccountToken: false si non nécessaire
■	Secrets depuis Azure Key Vault via External Secrets Operator (pas de secrets en dur dans YAML)
■	Aucun secret ni mot de passe dans ConfigMap ou variables d'environnement en clair
■	NetworkPolicy: deny-all par défaut + whitelist explicite
■	PodDisruptionBudget (PDB) défini pour les Deployments en production
■	imagePullPolicy: Always ou tag fixe (jamais latest en prod)
■	revisionHistoryLimit: 5 (minimum) pour permettre les rollbacks
■	Labels app, version, environment sur tous les objets
■	ResourceQuota et LimitRange définis sur chaque namespace de production

■ Valider vos manifests avant de les appliquer : `kubectl apply --dry-run=client -f manifest.yaml` · `kubeval manifest.yaml` (outil tiers) · `kube-score manifest.yaml` (score de bonnes pratiques) · Différer les changements : `kubectl diff -f manifest.yaml`

PS-4 — Service, Ingress et Vérification réseau

PS-5 — Créer et Vérifier le RBAC

PS-6 — HPA, NetworkPolicy, Job et Diagnostics

PS-7 — Tableau récapitulatif : commandes PowerShell par objet K8s

Objet K8s	Commande PowerShell / kubectl via PS	Résultat
Pod	<pre>kubectl get pods -n prod kubectl describe pod -n prod kubectl logs -f -n prod --tail=100 kubectl exec -it -n prod -- /bin/sh</pre>	Lister / détailler / logs / shell interactif
Deployment	<pre>kubectl apply -f deploy.yaml wait-Rollout -Deployment mon-app Undo-DeploymentRollout -Name mon-app Scale-Deployment -Name mon-app -Replicas 5</pre>	Appliquer / attendre / rollback / scaler
Service	<pre>kubectl get svc -n prod -o wide Expose-Deployment -Name mon-app -Type ClusterIP Start-PortForward -Service mon-app-svc -LocalPort 8080 -RemotePort 80</pre>	Lister / créer / tunnel local
Ingress	<pre>kubectl get ingress -n prod New-Ingress -Name app-ingress -Host api.local kubectl describe ingress app-ingress -n prod</pre>	Lister / créer avec TLS / détailler
ConfigMap	<pre>New-ConfigMap -Name app-cfg -Data @{ENV='prod'} kubectl get cm -n prod kubectl edit cm app-cfg -n prod</pre>	Créer / lister / éditer
Secret	<pre>New-K8sSecret -Name app-sec -Data @{'pw'='motdepasse'} Get-SecretValue -Name app-sec -Key pw Get-AllSecrets -Namespace prod</pre>	Créer / lire / lister
RBAC	<pre>New-AppRBAC -AppName mon-app -Namespace prod Test-SAPermissions -ServiceAccount mon-app-sa Get-UserRoles -Subject mon-app-sa</pre>	Créer SA+Role+Binding / auditer / lister
HPA	<pre>New-HPA -DeploymentName mon-app -MinReplicas 3 -MaxReplicas 20 kubectl get hpa -n prod kubectl describe hpa mon-app-hpa -n prod</pre>	Créer / lister / détailler
NetworkPolicy	<pre>Set-NetworkPolicyDenyAll -Namespace prod kubectl get networkpolicies -n prod Test-NetworkFromPod -Target postgres-svc -Port 5432</pre>	Bloquer tout / lister / tester connectivité
Job	<pre>Invoke-K8sJob -Name migrate -Image app:v1 -Command @('python','migrate.py') kubectl get jobs -n prod kubectl logs job/migrate -n prod</pre>	Lancer / suivre / voir logs
Diagnostics	<pre>Get-NamespaceHealth -Namespace prod Invoke-RollingRestart -Name mon-app Apply-Manifest -Path ./k8s/ -ShowDiff -DryRun</pre>	Vue d'ensemble / restart / dry-run+diff

PARTIE 3 — Helm & Helm Charts

Helm est le gestionnaire de paquets Kubernetes. Un Chart Helm est un ensemble de templates YAML paramétrables regroupés en un artefact versionnable. Structure, values.yaml, templates Go, hooks, dépendances et CLI helm complète.

3.1 — Structure d'un Chart Helm

```
mon-chart/ ■■■ Chart.yaml # Metadonnées (obligatoire) ■■■ values.yaml # Valeurs par défaut ■■■ values.schema.json
# Schema JSON validation (optionnel) ■■■ .helmignore ■■■ charts/ # Dépendances ■■■ templates/ ■■■ _helpers.tpl
# Fonctions réutilisables ■■■ deployment.yaml ■■■ service.yaml ■■■ ingress.yaml ■■■ configmap.yaml ■
■■■ secret.yaml ■■■ hpa.yaml ■■■ serviceaccount.yaml ■■■ NOTES.txt # Message après helm install ■■■
tests/test-connection.yaml ■■■ crds/ # CRDs (installés en premier)
```

Chart.yaml + values.yaml essentiels

```
# Chart.yaml apiVersion: v2 name: mon-app type: application version: 1.3.0 appVersion: "2.1.4" dependencies: - name:
postgresql version: "15.x.x" repository: https://charts.bitnami.com/bitnami condition: postgresql.enabled --- #
values.yaml (extrait) replicaCount: 3 image: repository: registry.example.com/mon-app pullPolicy: Always tag: "" #
Surcharge par --set image.tag=1.2.0 service: type: ClusterIP port: 80 targetPort: 8080 ingress: enabled: true
className: nginx annotations: cert-manager.io/cluster-issuer: letsencrypt-prod hosts: - host: api.example.com paths:
- path: / pathType: Prefix tls: - secretName: app-tls hosts: [api.example.com] resources: limits: { cpu: 500m,
memory: 512Mi } requests: { cpu: 250m, memory: 256Mi } autoscaling: enabled: true minReplicas: 3 maxReplicas: 20
targetCPUUtilizationPercentage: 70 postgresql: enabled: false
```

Template deployment.yaml (syntaxe Go template)

```
apiVersion: apps/v1 kind: Deployment metadata: name: {{ include "mon-app.fullname" . }} namespace: {{
.Release.Namespace }} labels: {{- include "mon-app.labels" . | nindent 4 }} spec: {{- if not
.Values.autoscaling.enabled }} replicas: {{ .Values.replicaCount }} {{- end }} selector: matchLabels: {{- include
"mon-app.selectorLabels" . | nindent 6 }} template: metadata: annotations: checksum/config: {{ include (print
$.Template.BasePath "/configmap.yaml") . | sha256sum }} {{- with .Values.podAnnotations }} {{- toYaml . | nindent 8
}} {{- end }} labels: {{- include "mon-app.selectorLabels" . | nindent 8 }} version: {{ .Values.image.tag | default
.Chart.AppVersion | quote }} spec: serviceAccountName: {{ include "mon-app.serviceAccountName" . }} securityContext:
{{- toYaml .Values.podSecurityContext | nindent 8 }} containers: - name: {{ .Chart.Name }} image: "{{{
.Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}" imagePullPolicy: {{
.Values.image.pullPolicy }} ports: - name: http containerPort: {{ .Values.service.targetPort }} env: - name:
DB_PASSWORD valueFrom: secretKeyRef: name: {{ include "mon-app.fullname" . }}-secrets key: db-password
livenessProbe: {{- toYaml .Values.livenessProbe | nindent 10 }} resources: {{- toYaml .Values.resources | nindent 10
}}
```

_helpers.tpl + Hook pre-upgrade

```
# _helpers.tpl {{- define "mon-app.fullname" -}} {{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 |
trimSuffix "-" }} {{- end }} {{- define "mon-app.labels" -}} helm.sh/chart: {{ printf "%s-%s" .Chart.Name
.Chart.Version | replace "+" "-" }} {{ include "mon-app.selectorLabels" . }} app.kubernetes.io/version: {{
.Chart.AppVersion | quote }} app.kubernetes.io/managed-by: {{ .Release.Service }} {{- end }} {{- define
"mon-app.selectorLabels" -}} app.kubernetes.io/name: {{ .Chart.Name }} app.kubernetes.io/instance: {{ .Release.Name
}} {{- end }} --- # templates/job-migration.yaml – Hook pre-upgrade apiVersion: batch/v1 kind: Job metadata: name: {{
include "mon-app.fullname" . }}-migrate annotations: "helm.sh/hook": pre-upgrade,pre-install "helm.sh/hook-weight":
"-5" "helm.sh/hook-delete-policy": hook-succeeded spec: backoffLimit: 3 template: spec: restartPolicy: OnFailure
containers: - name: migrate image: "{{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion
}}}" command: ["python", "manage.py", "migrate", "--noinput"]
```

Hook	Moment d'exécution	Cas d'usage
pre-install	Avant la 1re installation	Ressources prerequisites
post-install	Après la 1re installation	Tests initiaux
pre-upgrade	Avant chaque mise à jour	Migration BDD
post-upgrade	Après mise à jour	Smoke tests
pre-delete	Avant la suppression	Archivage

Hook	Moment d'execution	Cas d'usage
test	Via helm test	Tests integration

3.2 — Toutes les commandes Helm CLI

```
# ===== REPOS ===== helm repo add bitnami
https://charts.bitnami.com/bitnami helm repo add jetstack https://charts.jetstack.io helm repo add argo
https://argoproj.github.io/argo-helm helm repo update # Mettre a jour les repos helm repo list # Lister helm search
repo nginx # Chercher un chart helm search hub postgresql # Chercher sur Artifact Hub # ===== INSTALL / UPGRADE
===== helm install mon-release ./mon-chart -n production
--create-namespace helm upgrade --install mon-release ./mon-chart -n prod # Install OU upgrade helm upgrade
mon-release ./mon-chart -f values-prod.yaml -f values-secrets.yaml --set image.tag=1.3.0 --set-string
"config.env=production" --atomic \ # Rollback auto si echec --wait \ # Attendre Ready --timeout 5m # Simulation et
debug helm install mon-app ./chart --dry-run --debug helm template mon-release ./mon-chart -f values-prod.yaml #
Rendu local helm lint ./mon-chart -f values-prod.yaml # Verifier syntaxe # ===== INSPECTER
===== helm list -n production # Releases installees helm
list -A # Toutes les namespaces helm status mon-release -n prod # Statut helm history mon-release -n prod #
Historique revisions helm get values mon-release -n prod # Values utilisees helm get manifest mon-release -n prod #
YAML applique final helm get notes mon-release -n prod # NOTES.txt # ===== DIFF avant upgrade (plugin)
===== helm plugin install https://github.com/databus23/helm-diff helm diff
upgrade mon-release ./mon-chart -f values-prod.yaml # ===== ROLLBACK / UNINSTALL
===== helm rollback mon-release 3 -n prod # Revenir revision 3 helm
rollback mon-release 0 -n prod # Revision precedente helm uninstall mon-release -n prod helm uninstall mon-release -n
prod --keep-history # ===== DEPENDANCES ===== helm dependency
update ./mon-chart # Telecharger deps helm dependency list ./mon-chart helm package ./mon-chart # Creer archive .tgz
```

■ Toujours `helm diff + --dry-run` avant `helm upgrade`. Utiliser `--atomic` pour `rollback` automatique si echec. Plugin `helm-secrets` (SOPS) pour chiffrer les fichiers `values secrets`.

Helm + PowerShell

```
function Install-HelmChart { param( [string]$ReleaseName, [string]$Chart, [string]$Namespace="production",
[string[]]$ValuesFiles=@(), [hashtable]$SetValues=@{}, [switch]$Atomic, [switch]$DryRun, [string]$Timeout="5m" )
$Args = @("upgrade","--install",$ReleaseName,$Chart,"-n",$Namespace, "--create-namespace","--timeout",$Timeout) if
($Atomic) { $Args += "--atomic" } if ($DryRun) { $Args += "--dry-run"; $Args += "--debug" } else { $Args += "--wait"
} foreach ($f in $ValuesFiles) { $Args += @("-f",$f) } foreach ($kv in $SetValues.GetEnumerator()) { $Args +=
@("--set","$(($kv.Key)=$(($kv.Value)))" ) Write-Host "[HELM] $ReleaseName <- $Chart" -ForegroundColor Cyan & helm @args
if ($LASTEXITCODE -ne 0) { Write-Error "[HELM] Echec : $ReleaseName" } else { Write-Host "[OK] $ReleaseName deploye"
-ForegroundColor Green } } function Get-HelmReleases { param([string]$Namespace="") $nsFlag = if ($Namespace) {
@("-n",$Namespace) } else { @("-A") } helm list @nsFlag -o json | ConvertFrom-Json | Select-Object
name,namespace,revision,status,chart,app_version | Format-Table -AutoSize } function Diff-HelmRelease {
param([string]$Release, [string]$Chart, [string[]]$ValuesFiles=@(), [string]$Namespace="production") $vf =
$ValuesFiles | ForEach-Object { @("-f",$_) } helm diff upgrade $Release $Chart @vf -n $Namespace } # Deployer la
stack ShipClass function Deploy-ShipClassStack { param([string]$Namespace="production", [string]$Tag="latest",
[switch]$DryRun) @( @{ Name="cert-manager"; Chart="jetstack/cert-manager"; Values=@{"installCRDs"="true"} }, @{
Name="ingress-nginx"; Chart="ingress-nginx/ingress-nginx"; Values=@{} }, @{ Name="vessel-service";
Chart="./charts/vessel-service"; Values=@{"image.tag"=$Tag} } ) | ForEach-Object { Install-HelmChart -ReleaseName
$_.Name -Chart $_.Chart -Namespace $Namespace -SetValues $_.Values -Atomic:(!$DryRun) -DryRun:$DryRun } } #
Install-HelmChart -ReleaseName "mon-app" -Chart "./charts/mon-app" ` # -ValuesFiles @("values-prod.yaml") -SetValues
@{"image.tag"="1.3.0"} -Atomic
```

PARTIE 4 — kubectl — Reference Complete

Contextes, GET avance avec formats de sortie, LOGS, EXEC, PATCH, DEBUG, rollout, scale, wait, top, drain et astuces de productivité PowerShell.

4.1 — Configuration et contextes

```
kubectl config get-contexts # Lister les contextes kubectl config current-context # Contexte actif kubectl config use-context mon-cluster-prod # Switcher kubectl config set-context --current --namespace=production # Fusionner kubeconfigs # PowerShell : $env:KUBECONFIG = "$HOME/.kube/config;$HOME/.kube/config-aks" # Linux/macOS : export KUBECONFIG=~/.kube/config:~/.kube/config-aks # Alias dans $PROFILE PowerShell Set-Alias k kubectl function kgp { kubectl get pods -n $args } function kgs { kubectl get svc -n $args } function kd { kubectl describe $args } function kl { kubectl logs -f $args } function ke { kubectl exec -it $args -- /bin/sh }
```

4.2 — GET avance — filtres, formats de sortie

```
# Filtres kubectl get pods -n prod -l app=mon-app # Par label kubectl get pods -n prod -l 'app in (a,b)' # Plusieurs valeurs kubectl get pods -n prod --field-selector status.phase=Running kubectl get pods -A # Tous namespaces # Formats de sortie kubectl get pod mon-pod -o yaml # YAML complet kubectl get pod mon-pod -o json # JSON complet kubectl get pods -o wide # + IP, noeud, image kubectl get pods -o custom-columns="NOM:.metadata.name,IMAGE:.spec.containers[0].image" kubectl get pods -o jsonpath='{.items[*].metadata.name}' kubectl get pods -o jsonpath='{range.items[*]}{.metadata.name}{ "}{.status.phase}{ " "}{end}' # Ressources multiples et tri kubectl get pods,svc,ingress -n production kubectl get all -n production kubectl api-resources # Tous les types disponibles kubectl get pods -n prod --sort-by='.status.startTime' kubectl get events -n prod --sort-by='.lastTimestamp' --field-selector type=Warning
```

4.3 — DESCRIBE / LOGS / EXEC / PORT-FORWARD

```
# DESCRIBE kubectl describe pod mon-pod -n prod # Pod (events, probes, limits) kubectl describe deployment mon-app -n prod # Deployment (conditions) kubectl describe certificate app-tls -n prod # Certificat TLS cert-manager kubectl describe node aks-node1 # Noeud (capacite, pods) # LOGS kubectl logs -f mon-pod -n prod # Follow kubectl logs -f mon-pod -c mon-container -n prod # Conteneur spécifique kubectl logs mon-pod --tail=100 -n prod # 100 dernières lignes kubectl logs mon-pod --since=1h -n prod # Dernière heure kubectl logs mon-pod --previous -n prod # Conteneur crashe kubectl logs -l app=mon-app -n prod # Tous les pods du label kubectl logs deployment/mon-app -n prod # Via le deployment # EXEC kubectl exec -it mon-pod -n prod -- /bin/bash # Shell interactif kubectl exec -it mon-pod -n prod -- /bin/sh # Alpine kubectl exec mon-pod -n prod -- env | sort # Variables d'env kubectl exec mon-pod -n prod -- curl -s http://localhost:8080/health kubectl exec -it mon-pod -n prod -- wget -qO- http://postgres-svc:5432 # PORT-FORWARD kubectl port-forward svc/mon-svc 8080:80 -n prod # Service (recommande) kubectl port-forward pod/mon-pod 8080:8080 -n prod # Pod direct kubectl port-forward deploy/mon-app 8080:8080 -n prod # Via deployment
```

4.4 — APPLY / EDIT / PATCH / DELETE / ROLLOUT / DEBUG

```
# APPLY kubectl apply -f manifest.yaml kubectl apply -f ./k8s/ -R # Récursif kubectl apply --dry-run=client -f manifest.yaml # Simulation locale kubectl apply --dry-run=server -f manifest.yaml # Validation API kubectl diff -f manifest.yaml # Diff avant apply # EDIT / PATCH kubectl edit deployment mon-app -n prod # Editeur interactif # $env:KUBE_EDITOR = "code --wait" # VS Code comme editeur kubectl patch deployment mon-app -n prod --type=json -p='[{"op":"replace","path":"/spec/replicas","value":5}]' kubectl patch deployment mon-app -n prod --type=merge -p='{ "spec": { "template": { "spec": { "terminationGracePeriodSeconds": 60 } } } }' # SET (raccourcis) kubectl set image deployment/mon-app app=reg/app:1.4.0 -n prod kubectl set env deployment/mon-app LOG_LEVEL=debug -n prod kubectl set resources deployment/mon-app -c app --limits=cpu=1,memory=1Gi --requests=cpu=500m,memory=512Mi -n prod # DELETE kubectl delete -f manifest.yaml kubectl delete pod mon-pod -n prod --grace-period=0 --force kubectl delete pods -l app=mon-app -n prod kubectl delete all -l app=mon-app -n prod # ROLLOUT kubectl rollout restart deployment/mon-app -n prod # Rolling restart kubectl rollout status deployment/mon-app -n prod # Suivre kubectl rollout history deployment/mon-app -n prod # Historique kubectl rollout undo deployment/mon-app -n prod # Rollback precedent kubectl rollout undo deployment/mon-app --to-revision=3 -n prod # SCALE / HPA kubectl scale deployment/mon-app --replicas=5 -n prod kubectl autoscale deployment/mon-app --cpu-percent=70 --min=3 --max=20 -n prod # DEBUG / DIAGNOSTICS kubectl top pods -n prod --sort-by=memory # CPU/RAM pods kubectl top nodes # CPU/RAM noeuds kubectl debug -it mon-pod --image=nicolaka/netshoot --target=mon-app -n prod kubectl run dbg --rm -it --restart=Never --image=nicolaka/netshoot -n prod -- bash kubectl wait --for=condition=ready pod -l app=mon-app -n prod --timeout=60s kubectl wait --for=condition=complete job/migrate -n prod --timeout=300s kubectl auth can-i create pods -n prod kubectl auth can-i '*' '*' --all-namespaces kubectl cordon aks-node1 # Bloquer nouveaux pods kubectl drain aks-node1 --ignore-daemonsets --delete-emptydir-data
```


PARTIE 5 — oc CLI — OpenShift Container Platform

oc est la CLI d'OpenShift, superset de kubectl. Commandes spécifiques : Projects, Routes, BuildConfigs, ImageStreams, DeploymentConfigs, SCC (Security Context Constraints) et administration OCP.

5.1 — Connexion, projets et statut

```
# CONNEXION oc login https://api.cluster.example.com:6443 # Login interactif oc login
https://api.cluster.example.com:6443 --token=sha256-xxxxx oc login https://api.cluster.example.com:6443 -u admin -p
pwd oc whoami # Utilisateur actif oc whoami --show-console # URL console web oc whoami -t # Token actif oc logout #
PROJECTS (namespaces OCP enrichis) oc projects # Tous les projets oc project production # Switcher oc new-project
staging --display-name="Staging" --description="Pre-prod" oc delete project staging oc status # Vue d'ensemble
projet actif oc status -v # Verbeux (warnings inclus)
```

5.2 — Routes (equivalent Ingress OpenShift)

```
# Routes simples oc expose svc/mon-app # HTTP simple oc expose svc/mon-app --hostname=api.apps.cluster.local #
Hostname custom # Manifest Route TLS apiVersion: route.openshift.io/v1 kind: Route metadata: name: mon-app-route
namespace: production spec: host: api.apps.cluster.local to: kind: Service name: mon-app-svc weight: 100 port:
targetPort: http tls: termination: edge # edge | passthrough | reencrypt insecureEdgeTerminationPolicy: Redirect
wildcardPolicy: None # Commandes oc get routes -n production oc delete route mon-app-route -n production oc describe
route mon-app-route -n production
```

5.3 — Builds, ImageStreams, S2I et DeploymentConfig

```
# NEW-APP (S2I — Source-to-Image) oc new-app python:3.9-https://github.com/example/app.git # Depuis Git oc new-app
--docker-image=registry.io/app:latest # Depuis image oc new-app --template=postgresql-persistent -p
POSTGRES_USER=admin -p POSTGRES_PASSWORD=secret # BUILDS oc start-build mon-app # Lancer oc start-build mon-app
--from-file=./app.tar # Depuis archive oc start-build mon-app --follow # Suivre logs oc get builds -n production oc
logs build/mon-app-3 -n production # IMAGESTREAMS oc get imagestreams -n production oc import-image mon-app:latest
--from=registry.io/mon-app:latest --confirm -n prod oc tag registry.io/app:1.2.0 production/mon-app:stable oc
describe imagestream mon-app -n production # DEPLOYMENTCONFIG (DC, legacy OCP avant Deployment apps/v1) oc rollout
latest dc/mon-app -n prod # Forcer rollout oc rollout status dc/mon-app -n prod # Statut oc rollout undo dc/mon-app
-n prod # Rollback oc scale dc/mon-app --replicas=5 -n prod oc set image dc/mon-app app=registry.io/app:1.3.0 -n prod
oc get dc -n production
```

5.4 — SCC (Security Context Constraints) et Administration OCP

```
# SCC — SECURITE OPENSHIFT oc get scc # Lister les SCC oc describe scc restricted-v2 # Details oc adm policy who-can
use scc anyuid # Assigner SCC a un ServiceAccount oc adm policy add-scc-to-user anyuid -z mon-app-sa -n production oc
adm policy add-scc-to-user privileged -z mon-app-sa -n production # ATTENTION ! oc adm policy remove-scc-from-user
anyuid -z mon-app-sa -n production # Verifier SCC d'un pod oc get pod mon-pod -n prod -o
jsonpath='{.metadata.annotations.openshift\.io/scc}' # ADMINISTRATION OCP oc adm top nodes # CPU/RAM noeuds oc adm
top pods -n production oc adm node-logs aks-node1 --unit=kubelet # Logs kubelet oc adm must-gather # Collecter infos
debug oc adm upgrade # Verifier MAJ OCP oc adm upgrade --to-latest # Upgrade oc adm policy add-cluster-role-to-user
cluster-admin admin
```

SCC	Caractéristique	Usage
restricted-v2	Interdit root, UID aleatoire, caps limitees	Default OCP 4.11+
restricted	Ancien default (OCP < 4.11)	Legacy
anyuid	N'importe quel UID, pas privileged	Images avec UID fixe
nonroot	Tout UID non-root, flexible	Apps sans UID precis
hostnetwork	Acces reseau et ports du noeud	Ingress, monitoring
privileged	Acces complet (root, devices, hostPath)	Systeme seulement !

5.5 — oc + PowerShell

```
function Connect-OpenShift { param([string]$ApiUrl, [string]$Token="", [string]$User="", [string]$Pass="") if
($Token) { oc login $ApiUrl --token=$Token --insecure-skip-tls-verify } else { oc login $ApiUrl -u $User -p $Pass
--insecure-skip-tls-verify } Write-Host "[OCP] Connecte : $(oc whoami)" -ForegroundColor Green } function
New-OCRoute { param([string]$ServiceName, [string]$Hostname="", [string]$Namespace="",
[ValidateSet("edge","passthrough","reencrypt")][string]$TLS="edge") $nsF = if ($Namespace) { "-n $Namespace" } else
{ "" } $hF = if ($Hostname) { "--hostname=$Hostname" } else { "" } Invoke-Expression "oc expose svc/$ServiceName $hF
$nsF" $patch = "{`"spec`":{`"tls`":{`"termination`":`"$TLS`",`"insecureEdgeTerminationPolicy`":`"Redirect`"}}}"
Invoke-Expression "oc patch route/$ServiceName -p '$patch' $nsF" Write-Host "[OK] Route TLS ($TLS) pour
$ServiceName" -ForegroundColor Green } function Set-SCCSA { param([string]$ServiceAccount, [string]$Namespace,
[ValidateSet("anyuid","restricted","nonroot","privileged","hostnetwork")][string]$SCC="anyuid") oc adm policy
add-scc-to-user $SCC -z $ServiceAccount -n $Namespace Write-Host "[OK] SCC '$SCC' -> $ServiceAccount dans
$Namespace" -ForegroundColor Green } function Start-OCBuild { param([string]$BC, [string]$Namespace="",
[switch]$Follow) $nsF = if ($Namespace) { "-n $Namespace" } else { "" } $fF = if ($Follow) { "--follow" } else { "" }
Invoke-Expression "oc start-build $BC $fF $nsF" } # kubectl vs oc vs helm – Cheat Sheet # KUBECTL OC HELM # kubectl
create namespace ns = oc new-project ns --create-namespace # kubectl apply -f deploy.yaml = oc apply -f / oc new-app
helm install/upgrade # kubectl apply -f ingress.yaml = oc expose svc + patch tls Ingress dans chart # kubectl logs -f
pod = oc logs -f / oc rsh (shell) kubectl/oc apres install # kubectl rollout undo deploy = oc rollout undo dc helm
rollback 3 # kubectl scale deploy --rep=5 = oc scale dc --replicas=5 --set replicaCount=5 # kubectl create
rolebinding = oc adm policy add-role-to-user values.yaml serviceAccount # (aucun equivalent direct) = oc adm policy
add-scc-to-user securityContext values # Exemples : # Connect-OpenShift -ApiUrl "https://api.aro.cluster.com:6443"
-Token "sha256-..." # New-OCRoute -ServiceName "vessel-svc" -Hostname "api.apps.cluster.local" -TLS "edge" #
Set-SCCSA -ServiceAccount "vessel-sa" -Namespace "production" -SCC "anyuid" # Start-OCBuild -BC "mon-app" -Namespace
"production" -Follow
```

5.6 — Tableau comparatif kubectl / oc / helm

Action	kubectl	oc	helm
Creer namespace/projet	kubectl create namespace prod	oc new-project prod	--create-namespace
Deployer une app	kubectl apply -f deploy.yaml	oc apply -f oc new-app image:tag	helm install app ./chart
Exposer HTTPS	kubectl apply -f ingress.yaml (+ Ingress controller)	oc expose svc/app oc patch route tls	Ingress dans le chart + helm upgrade
Voir les logs	kubectl logs -f pod	oc logs -f pod oc rsh pod (shell)	kubectl/oc apres install
Mettre a jour image	kubectl set image deploy/app app=img:v2	oc set image dc/app app=img:v2	helm upgrade --set image.tag=v2
Rollback	kubectl rollout undo deploy/app	oc rollout undo dc/app	helm rollback app 3
Scaler	kubectl scale deploy/app --replicas=5	oc scale dc/app --replicas=5	helm upgrade --set replicaCount=5
Securite pod	PodSecurityStandards NetworkPolicy	SCC oc adm policy add-scc-to-user	securityContext dans values.yaml
Supprimer tout	kubectl delete all -l app=x	kubectl delete all -l app=x	helm uninstall app